

MPI Parallelization of MLP Training on MNIST

Final Project Report

Author One* Author Two* Author Three
[Course Name & Number, Institution]

May 2026

GitHub Repository: github.com/your-org/your-repo

Abstract

We study how far MPI-based parallelism can accelerate end-to-end neural-network training while preserving training quality under controlled settings. Starting from a serial C++17 MLP baseline on MNIST, we implement and compare four MPI execution modes: flat data parallelism, two-level hierarchical data parallelism, local-SGD with periodic synchronization, and layer-wise model parallelism. To keep comparisons fair, all backends use matched model shape, optimizer configuration, dataset subsets, and output metrics schema, and are launched with one thread per rank for BLAS/OpenMP runtimes. We also apply shared engineering optimizations such as memory reuse in hot paths and release compiler settings. The main empirical outcome is that communication structure, not only rank count, determines realized speedup: local-SGD gives the best raw epoch-time scaling at large cluster size, flat DP remains a strong and stable baseline, hierarchical DP does not outperform flat DP for the current model size, and model parallelism is most sensitive to partition balance and communication boundaries.

1 Hypothesis

We investigate whether communication-aware MPI training strategies can deliver better time-to-epoch than a standard flat all-reduce baseline while maintaining comparable optimization behavior on MNIST. Our hypothesis has two parts:

- communication pattern matters as much as rank count, so methods that reduce or amortize global synchronization should scale better at larger process counts;
- implementation details (memory reuse, consistent launch semantics, and stable timing definitions) are necessary to expose the true algorithmic differences across backends.

The observed results support this hypothesis. Local-SGD achieves the best large-scale epoch-time reduction, flat DP remains the strongest fully synchronous baseline, and hierarchical DP underperforms flat DP in our current workload regime because its extra synchronization phases are not sufficiently amortized by model size.

2 Context

Member contributions.

*Equal contribution.

- **Author One (replace with final name):** serial baseline, shared training utilities, and flat data-parallel baseline.
- **Author Two (replace with final name):** hierarchical DP and local-SGD variants, launch scripts, and scaling experiments.
- **Author Three (replace with final name):** model-parallel path, reporting pipeline, and methodology/results write-up.

Broader context. This project is a course-focused systems study rather than a direct sub-project of an external research lab artifact. Its emphasis is reproducible performance evaluation of parallel training designs under matched conditions.

Applicability to other courses. Variants of this project are immediately reusable in high-performance computing, distributed systems, and machine-learning systems courses because the same codebase supports serial baselining, collective-communication studies, point-to-point pipeline studies, and scaling-law analysis with consistent metrics outputs.

3 Prior Work

Work 1. [Sergeev and Del Balso \[2018\]](#) established a practical architecture for synchronous distributed data-parallel training that relies on collective communication and minimal intrusion into the training loop. Our flat-DP baseline follows this same principle: local gradient computation plus global aggregation each step, with careful control of launch and threading semantics.

Work 2. [Stich \[2019\]](#) showed that periodic synchronization can significantly reduce communication while retaining useful convergence behavior. Our local-SGD mode adopts this idea by synchronizing fused model weights every K steps and quantifying the resulting speed/accuracy trade-off in a controlled MPI setting.

Work 3. [Narayanan et al. \[2019\]](#) demonstrated the importance of pipeline-aware scheduling in model-parallel training and highlighted bubble overhead as a first-order performance factor. Our current model-parallel implementation is non-pipelined but includes explicit pipeline placeholders and an alpha-beta communication model to prepare a clean extension path for GPipe/1F1B-style variants in Section 4.

4 Empirical Methodology

4.1 Problem Setup

We study parallel training of a feed-forward MLP on MNIST with a fixed optimization setup and matched data split across implementations. The primary model for scaling is $784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 10$, trained with SGD and cross-entropy. The project compares five execution modes:

- **Serial:** single-process baseline.
- **Flat MPI Data Parallel (DP):** each rank computes local gradients, then performs gradient all-reduce every step.
- **Hierarchical MPI DP:** per-step gradient synchronization split into intra-node reduce, inter-node all-reduce (leaders), and intra-node broadcast.
- **Local SGD:** each rank performs local SGD and averages weights every K steps.

- **Model Parallel (MP):** layers are partitioned across ranks and activations/gradients are exchanged between neighboring partitions.

Data-parallel workload. For serial and DP comparisons, we use the main scaling model $784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 10$, fixed optimizer settings, and a rank ladder from $1n4r$ to $4n128r$. This is the primary workload used for strong-scaling conclusions.

Model-parallel workload. For MP studies, we use a deeper architecture with 8 weight matrices $784 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10$ so layers can be distributed meaningfully across ranks. MP/pipeline experiments therefore analyze communication-scheduling behavior under a workload tailored to layer partitioning.

4.2 Sequential Baseline

The sequential baseline uses the same MLP, loss, and optimizer as all MPI variants. It runs a shuffled mini-batch loop over training data and reports `epoch_time_ms` as *training-loop-only* wall time (validation evaluation is measured but excluded from epoch timing). This timing definition is intentionally mirrored in the MPI backends to keep scaling comparisons consistent.

4.3 Parallel Design

4.3.1 Design Overview

Why multiple algorithms are shown. The implementations do not share one execution pattern. Serial training has no communication. Data-parallel methods synchronize gradients or weights with collectives. Model parallelism partitions layers and exchanges intermediate tensors point-to-point. For clarity, we therefore present separate algorithms for (1) serial training, (2) data-parallel training, and (3) model-parallel training.

4.3.2 Data-Parallel Designs

Serial. Single-process SGD over shuffled mini-batches; no MPI calls.

Algorithm 1 Serial training epoch

Require: shuffled indices I , batch size B , parameters θ

Ensure: updated θ after one epoch

```

1: for  $t = 0, B, 2B, \dots$  do
2:   gather batch  $(x, y)$  from  $I$ 
3:    $g \leftarrow \nabla \ell(\theta; x, y)$ 
4:    $\theta \leftarrow \theta - \eta g$ 
5: end for
```

Flat DP. Each rank processes a disjoint shard of each global batch, computes local gradients, then uses `MPI_Allreduce` on layer gradients to form the global mean gradient before applying the update.

Hierarchical DP. Gradient synchronization is decomposed into three phases: `MPI_Reduce` within a node, `MPI_Allreduce` across node leaders, then `MPI_Bcast` within each node. This reduces inter-node participants but introduces extra synchronization phases.

Local SGD. Ranks execute local updates independently and synchronize model weights via one fused `MPI_Allreduce` every K steps ($K \in \{10, 50\}$ in this report). This trades statistical freshness for lower communication frequency.

4.3.3 Model-Parallel and Pipeline Designs

Model Parallel (MP). Layers are partitioned across ranks. Forward activations and backward gradients are exchanged via point-to-point MPI between adjacent layer partitions. In this report, MP is treated as a first-class method in the main project (not an auxiliary baseline). We currently evaluate a non-pipelined schedule, and we reserve explicit placeholders for pipelined MP variants:

- **Pipeline placeholder A (inference-style fill/drain):** TODO add timeline and throughput model for micro-batch pipeline.
- **Pipeline placeholder B (1F1B schedule):** TODO add 1F1B warmup/steady/cooldown schedule and bubble analysis.
- **Pipeline placeholder C (interleaved virtual stages):** TODO add interleaved stage assignment and expected memory trade-offs.

Algorithm 2 Data-parallel epoch (flat DP / hierarchical DP / local SGD)

Require: shuffled index list I , global batch size B , rank count P

Ensure: updated model parameters after one epoch

```

1: for global batch offset  $t = 0, B, 2B, \dots$  do
2:   gather local mini-batch  $(x_r, y_r)$  from  $I$ 
3:   compute local gradients  $g_r \leftarrow \nabla \ell(\theta; x_r, y_r)$ 
4:   if mode is Flat DP then
5:      $g \leftarrow \text{AllreduceMean}(g_r)$ 
6:   else if mode is Hierarchical DP then
7:      $g \leftarrow \text{IntraReduce} \rightarrow \text{InterAllreduce} \rightarrow \text{IntraBcast}$ 
8:   else if mode is Local SGD then
9:     set  $g \leftarrow g_r$ ; synchronize weights only every  $K$  steps
10:  end if
11:  apply SGD update with synchronized quantity (gradient or weight)
12: end for
```

Algorithm 3 Model-parallel epoch (non-pipelined, per-batch)

Require: layer partition per rank, batch size B

Ensure: updated local layer parameters on each rank

```

1: for each batch  $(x, y)$  do
2:   receive activation from previous rank (or use input if first rank)
3:   compute local forward over owned layers
4:   send activation to next rank (unless last rank)
5:   receive backward gradient from next rank (or compute from loss if last rank)
6:   compute local backward over owned layers
7:   send gradient to previous rank (unless first rank)
8:   update local parameters
9: end for
```

4.4 Communication Cost Model (Alpha-Beta)

We use the standard latency-bandwidth model:

$$T_{\text{msg}}(m) = \alpha + \beta m, \quad (1)$$

where α is startup latency, β is per-byte (or per-float) transfer cost, and m is message size.

Flat DP (per step). With all-reduce payload m_g , a ring-style approximation is:

$$T_{\text{flat}}(m_g) \approx 2(P-1)\alpha + 2\frac{P-1}{P}\beta m_g. \quad (2)$$

Hierarchical DP (per step). Let q be ranks per node and N be number of nodes ($P = Nq$). A simple two-level approximation (intra reduce+bcast, inter all-reduce among leaders) is:

$$T_{\text{hier}}(m_g) \approx \underbrace{2 \log q \alpha + 2\frac{q-1}{q}\beta m_g}_{\text{intra-node}} + \underbrace{2(N-1)\alpha + 2\frac{N-1}{N}\beta m_g}_{\text{inter-node}}. \quad (3)$$

This can outperform flat DP when expensive inter-node traffic is reduced and message size is large enough to amortize extra phases.

Local SGD (amortized per step). If weight synchronization payload is m_w and synchronization is every K steps:

$$T_{\text{local,step}} \approx T_{\text{compute}} + \frac{1}{K}T_{\text{sync}}(m_w), \quad (4)$$

which explains reduced communication overhead as K increases.

Model Parallel (non-pipelined). If activation payload per boundary is m_a , backward payload is m_b , and rank r owns local compute C_r :

$$T_{\text{mp,step}} \approx \sum_{r=1}^P (C_r + (\alpha + \beta m_a) + (\alpha + \beta m_b)). \quad (5)$$

The practical bottleneck is the slowest stage plus boundary communication.

Pipeline placeholders (to be completed experimentally). For L pipeline stages and M microbatches, fill-drain bubble fraction is:

$$\text{bubble}(L, M) = \frac{L-1}{M+L-1}. \quad (6)$$

4.5 Implementation Details

The system is implemented in C++17 with MPI collectives/point-to-point communication and BLAS-backed dense linear algebra. Key implementation choices that affect performance and reproducibility are:

- **BLAS acceleration:** matrix multiply and vector updates were moved to BLAS-backed kernels (`sgemm/saxpy/sgemv`) to reduce scalar-loop overhead.

- **Backend isolation:** serial, flat DP, hierarchical DP, local-SGD, and model-parallel paths are built as separate executables with dedicated launch scripts.
- **Memory reuse in hot loops:**
 - preallocated mini-batch feature/label buffers are reused across steps to avoid per-batch allocation churn;
 - gather paths use contiguous writes to improve locality;
 - gradient and workspace buffers are reused in the shared MLP training path.
- **Compiler configuration:** Release builds use `-O3`, with optional `-march=native` and optional IPO/LTO when supported.

Serial baseline optimization context. The serial path is a tuned baseline rather than an untuned reference: it uses the same BLAS-backed kernels, reuse-oriented buffer strategy, and release build configuration as the parallel code paths where applicable. This ensures that reported parallel speedups are measured against a reasonably optimized single-rank implementation, not against avoidable serial overheads.

Because model parallelism uses a separate `MPLayerSlice` forward/backward path, some MLP-internal workspace optimizations apply directly to serial and data-parallel modes, while MP performance is driven more by partitioning balance and point-to-point communication behavior.

4.6 Experimental Setup

4.6.1 Data-Parallel Setup

All DP strong-scaling sweeps were executed through Slurm `srun` launchers. Thread-level oversubscription was disabled by pinning: `OMP_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`. The primary DP sweep used:

- epochs = 10, global batch = 1024
- train/validation samples = 50,000/10,000
- learning rate = 0.03, seed = 42
- hidden layers = 256, 128, 64
- rank ladder: $1n4r \rightarrow 1n16r \rightarrow 1n32r \rightarrow 2n64r \rightarrow 4n128r$

Timing and summary outputs are generated from `results/scaling/*.csv` and `results/scaling/*.log` via `summarize_dp_scaling.py` and `make_scaling_visuals.py`.

Table 1: Software/runtime configuration used for DP scaling runs.

Component	Specification
Runtime launcher	Slurm <code>srun</code> + MPI
Threading policy	OMP/BLAS/MKL threads pinned to 1
Build mode	Release (<code>-O3</code> , optional <code>-march=native</code> , optional IPO)
Dataset	MNIST (local cache via <code>prepare_mnist.sh</code>)
Model for main sweep	MLP 256, 128, 64 hidden layers
Scaling configs	$1n4r, 1n16r, 1n32r, 2n64r, 4n128r$

4.6.2 Model-Parallel / Pipeline Setup

MP and pipeline experiments use a deeper model to expose partitioning and communication effects:

$$784 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10.$$

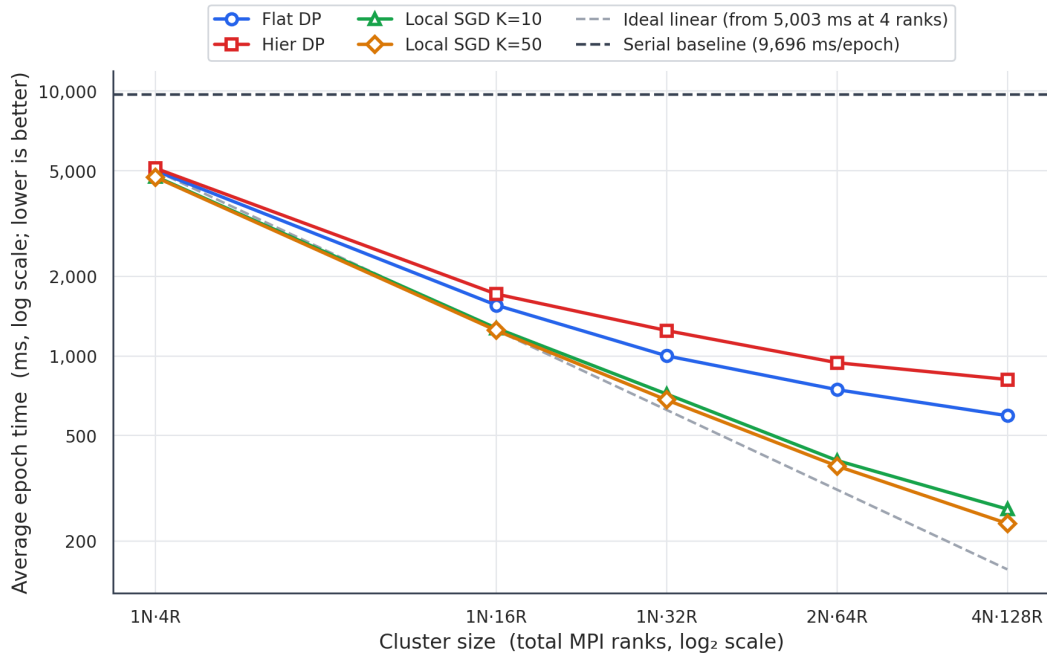
Unless otherwise noted, runs use 10 epochs, seed = 42, and the same thread-pinning policy as DP. The MP-specific sweeps are:

- **Batch-size sweep:** batch = 256 ... 4096, fixed world size = 8 with 2×4 layout (Sections 4.7.2).
- **Microbatch sweep:** fixed batch = 4096, world size = 8, varying microbatch count M (Section 4.7.2).
- **World-size sweep:** batch = 4096, $M = 32$, world size $ws \in \{2, 4, 6, 8\}$ with 2 tasks per node (Section 4.7.2).

4.7 Results

4.7.1 Data-Parallel Results

Scaling analysis. Figure 1 shows epoch time vs cluster size with serial baseline and ideal scaling reference. Local-SGD achieves the best time at 128 ranks ($K=50$ fastest), while flat DP remains more efficient than hierarchical DP for this model size and workload regime. A likely explanation is that `MPI_Allreduce` in the flat-DP path benefits from mature, topology-aware collective implementations in the system MPI stack, whereas the explicit two-level hierarchical sequence (intra-reduce, inter-allreduce, intra-bcast) adds extra synchronization phases that are not sufficiently amortized at the current gradient payload.



N = nodes, R = MPI ranks (e.g. 2N-64R = 2 nodes $\times$ 32 ranks each)

Figure 1: Epoch time vs cluster size with serial baseline and ideal line (MLP 256 $\rightarrow$ 128 $\rightarrow$ 64, MNIST, 10 epochs).

Strong scaling. Using flat DP at $1n4r$ as baseline (≈ 5003 ms/epoch), strong-scaling speedup and efficiency are:

Table 2: Strong scaling for flat DP (baseline: 1n4r).

Config	Time (ms)	Speedup $S(p)$	Efficiency $E(p)$
1n4r	5003	1.00×	100.0%
1n16r	1555	3.22×	80.4%
1n32r	1001	5.00×	62.5%
2n64r	746	6.71×	41.9%
4n128r	595	8.40×	26.3%

Compared to flat DP, hierarchical DP trails at all tested scales in the current runs, while Local-SGD surpasses flat DP speedup as K increases, reaching $21.46\times$ speedup at 4n128r ($K=50$) relative to the flat-DP 4-rank baseline.

Weak scaling. This report focuses on strong scaling. A dedicated weak-scaling sweep (proportionally increasing data size with rank count) is not yet included in the current artifact set and is left as future work.

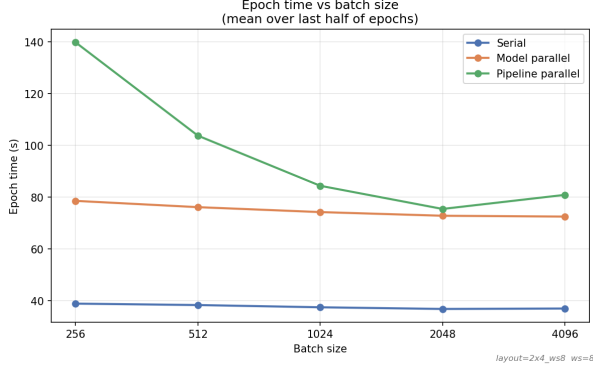
Additional DP observations. Timing breakdowns from logs show:

- flat DP: communication cost appears primarily in `allreduce_ms`, while `other_ms` remains non-negligible at high ranks. Combined with better end-to-end epoch times, this is consistent with (but does not by itself prove) topology-aware optimization of `MPI_Allreduce` by the MPI runtime; the exact per-call internal collective algorithm is not fully exposed in public documentation for the production MPI stack used on this system [NERSC, 2026a,b, HPE Cray Programming Environment, 2025].
- hierarchical DP: cost is split into `intra_reduce_ms`, `inter_allreduce_ms`, and `intra_bcast_ms`, confirming additional synchronization phases.
- local-SGD: `sync_ms` is much smaller than per-step DP comm cost because communication occurs only every K steps.

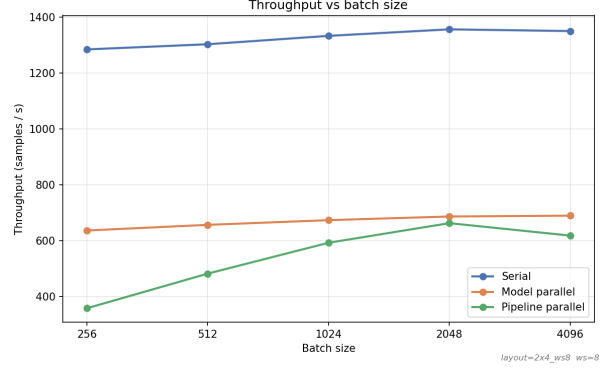
Accuracy trade-off is modest for the fastest local-SGD settings in these runs: at 128 ranks, $K=50$ is fastest with a small validation-accuracy drop relative to flat DP.

4.7.2 Model-Parallel Results

Epoch time and throughput vs. batch size. Figure 2 shows epoch time and throughput as batch size varies from 256 to 4096 at a fixed world size of 8 (layout 2×4) for serial, sequential model parallel (`mpi-mp`), and load-balanced pipeline parallel (`mpi-mp-pip`). The model used for these experiments is $784 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10$, chosen to have enough layers (8 weight matrices) to distribute meaningfully across 8 ranks.



(a) Epoch time vs. batch size.



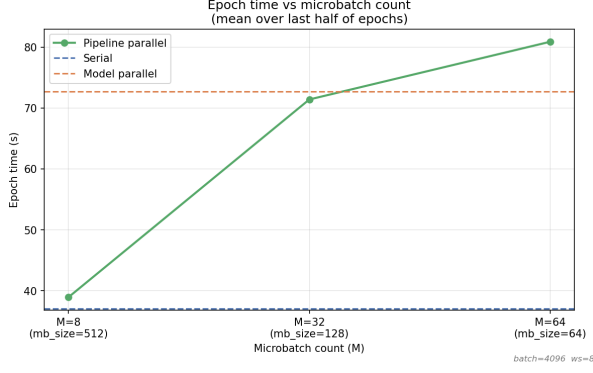
(b) Throughput vs. batch size.

Figure 2: Model parallelism performance vs. batch size (MLP 512→512→256→256→128→64→32, MNIST, 10 epochs, ws = 8, layout 2×4).

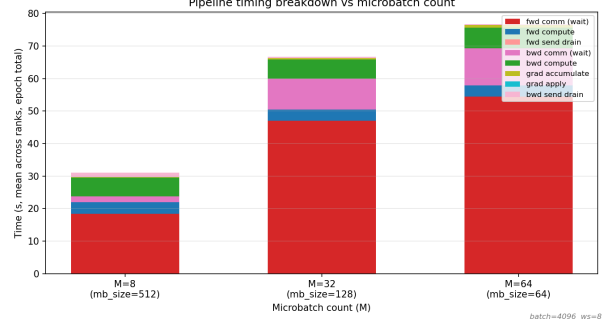
Several patterns are evident.

- **Both parallel modes are uniformly slower than serial.** Sequential model parallel runs at approximately 72–79 s/epoch across all batch sizes, roughly 2× the serial baseline of 37–39 s. This is expected: in non-pipelined model parallelism the total step time equals the *sum* of all ranks’ compute plus inter-rank communication, so adding more ranks moves work between processes but does not reduce wall time for this model size.
- **Pipeline parallel is strongly batch-size sensitive.** At batch = 256 pipeline epoch time is approximately 140 s (nearly 4× serial) because the microbatch size is only 8 samples per slot, making per-message latency the dominant cost. As batch size grows the microbatch size grows proportionally and pipeline time falls sharply, reaching ≈ 74 s at batch = 2048 before rising slightly to 81 s at batch = 4096. The modest increase at 4096 is consistent with the microbatch count rising from 32 to 64 at that point (see Section 4.7.2), which increases cumulative message latency.
- **Throughput gap is consistent.** Serial achieves 1,285–1,355 samples/s. Sequential MP achieves 635–690 samples/s (≈ 50% of serial). Pipeline throughput rises from 355 samples/s at batch = 256 to 660 samples/s at batch = 2048, converging toward MP at larger batches.

Microbatch count sensitivity. Figure 3 shows epoch time vs. microbatch count M at a fixed batch size of 4096 and ws = 8, together with the per-category timing breakdown for the pipeline runs.



(a) Epoch time vs. microbatch count M .



(b) Per-category timing breakdown (pipeline, mean across ranks).

Figure 3: Microbatch count sensitivity for pipeline parallelism (batch = 4096, ws = 8).

The result is counterintuitive: *fewer microbatches is faster*.

- At $M=8$ (microbatch size 512 rows) pipeline epoch time is ≈ 39 s, nearly matching serial (≈ 37 s) and substantially outperforming sequential MP (≈ 72 s).
- At $M=32$ (microbatch size 128) epoch time rises to ≈ 71 s, on par with sequential MP.
- At $M=64$ (microbatch size 64) epoch time is ≈ 81 s, the slowest of the three despite having the highest theoretical pipeline efficiency $M/(M+P-1) = 89\%$.

The timing breakdown (Figure 3b) explains this directly. Forward communication wait (`fwd_comm`) grows from ≈ 18 s at $M=8$ to ≈ 47 s at $M=32$ and ≈ 55 s at $M=64$, growing roughly linearly with M . Each microbatch requires a separate MPI point-to-point send across the activation boundary, so cumulative latency accumulates as $M \cdot (\alpha + \beta m_a)$. For MNIST activations the tensor is small (e.g. 512×512 floats at $M=8$, smaller still at higher M), meaning startup cost α dominates over bandwidth cost βm_a . Increasing M therefore multiplies latency proportionally without a corresponding increase in useful compute per message, overwhelming the theoretical bubble-fraction benefit that motivates larger M values.

Scaling with world size. Figure 4 shows epoch time vs. world size at batch = 4096 and $M=32$, with 2 tasks per node across configurations $ws \in \{2, 4, 6, 8\}$.

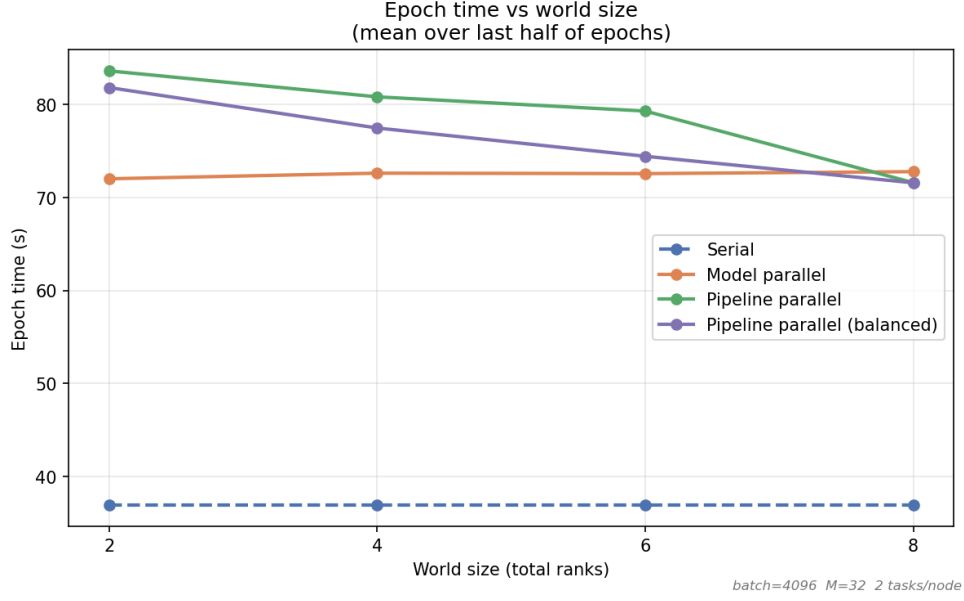


Figure 4: Epoch time vs. world size for all model-parallel modes (batch = 4096, $M=32$, 2 tasks/node). Serial shown as dashed reference at ≈ 37 s.

- **Sequential model parallel does not improve with world size.** Epoch time is essentially flat at $\approx 72\text{--}73$ s across all tested world sizes. This is the expected behavior: the non-pipelined schedule serializes all ranks, so adding more ranks subdivides the compute but adds proportional communication hops, leaving total wall time unchanged.
- **Pipeline parallel shows modest improvement with world size.** Unbalanced pipeline drops from ≈ 84 s at $ws=2$ to ≈ 79.5 s at $ws=8$. Load-balanced pipeline improves more consistently, from ≈ 82 s at $ws=2$ to ≈ 71.5 s at $ws=8$, converging toward the sequential MP baseline at the largest world size tested.
- **Serial remains faster than all parallel variants at every world size.** At ≈ 37 s/epoch the serial baseline sits well below all model-parallel configurations. Throughput (Figure 5) confirms this: serial achieves $\approx 1,345$ samples/s vs. ≈ 700 samples/s for the best parallel variant at $ws=8$.

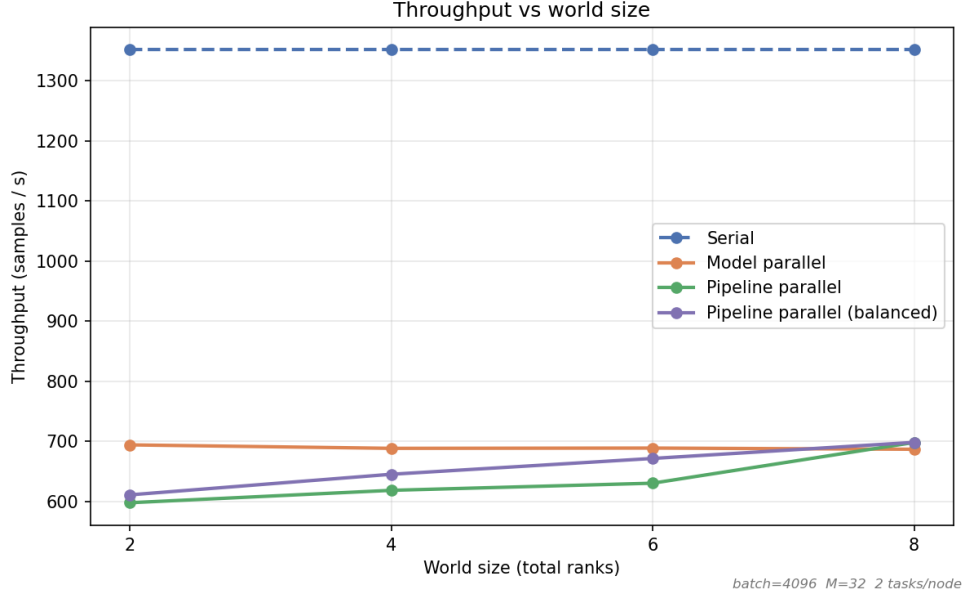


Figure 5: Throughput vs. world size (batch = 4096, $M = 32$, 2 tasks/node).

Load-balanced layer assignment. The architecture $784 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10$ has highly skewed layer costs: the first weight matrix (784×512 , $\approx 401,000$ parameters) accounts for roughly 45% of total computation, while the last three layers together account for under 1%. Naive round-robin assignment concentrates this imbalance on rank 0 while later ranks sit idle at each pipeline stage boundary. The load-balanced partitioner distributes layers greedily to equalize total parameter count per rank.

Figure 6 quantifies the improvement as $(t_{\text{pip}} - t_{\text{bal}})/t_{\text{pip}} \times 100$, and Table 3 summarizes the values.

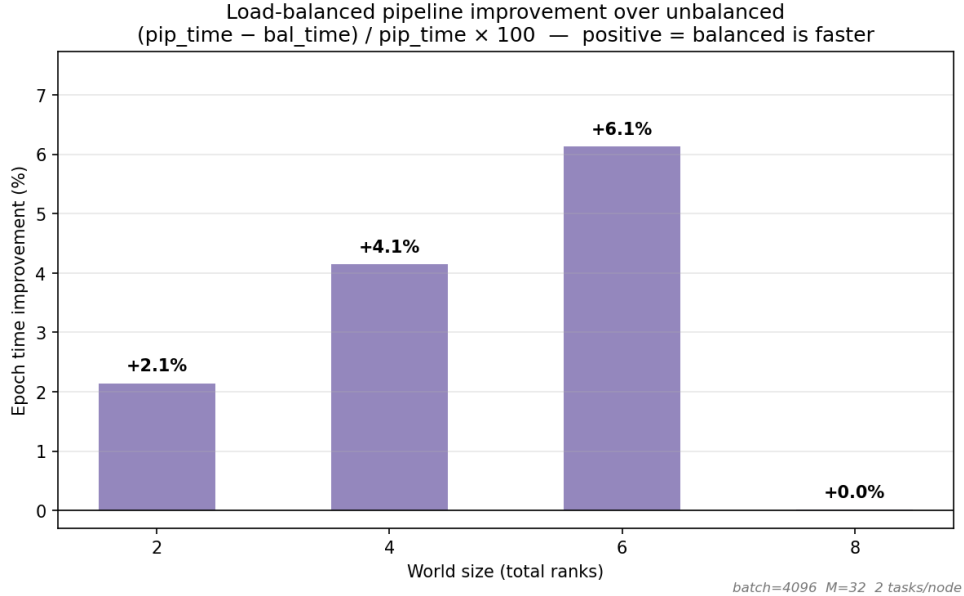


Figure 6: Load-balanced pipeline improvement over naive round-robin assignment (batch = 4096, $M=32$, 2 tasks/node). Positive values indicate the balanced partitioner is faster. Zero at $ws=8$ is expected: with 8 layers and 8 ranks every rank owns exactly one layer under both strategies.

Table 3: Load-balanced pipeline improvement over naive layer assignment by world size (batch = 4096, $M=32$, 2 tasks/node).

ws	Unbalanced (s)	Balanced (s)	Improvement
2	≈ 84.0	≈ 82.2	+2.1%
4	≈ 80.8	≈ 77.5	+4.1%
6	≈ 79.4	≈ 74.5	+6.1%
8	≈ 71.5	≈ 71.5	+0.0%

Improvements increase monotonically with world size up to $ws=6$, peaking at +6.1%. The peak at $ws=6$ reflects the worst-case imbalance for the naive partitioner: 8 layers across 6 ranks forces 2 ranks to hold 2 layers each, and naive assignment places both heavy layers on the earliest ranks. The balanced partitioner selects which layers are grouped based on cost rather than rank index. At $ws=8$ each rank holds exactly one layer under both strategies, so the result is identical at +0.0%.

Summary: model parallelism vs. serial. Across all configurations tested, model parallelism does not surpass the serial baseline for this workload. Speedup relative to serial remains at approximately $0.5\times$ for all world sizes and both pipeline strategies (Figure 7), confirming that communication overhead and the sequential nature of layer-wise activation exchange dominate over any compute savings from distributing the model.

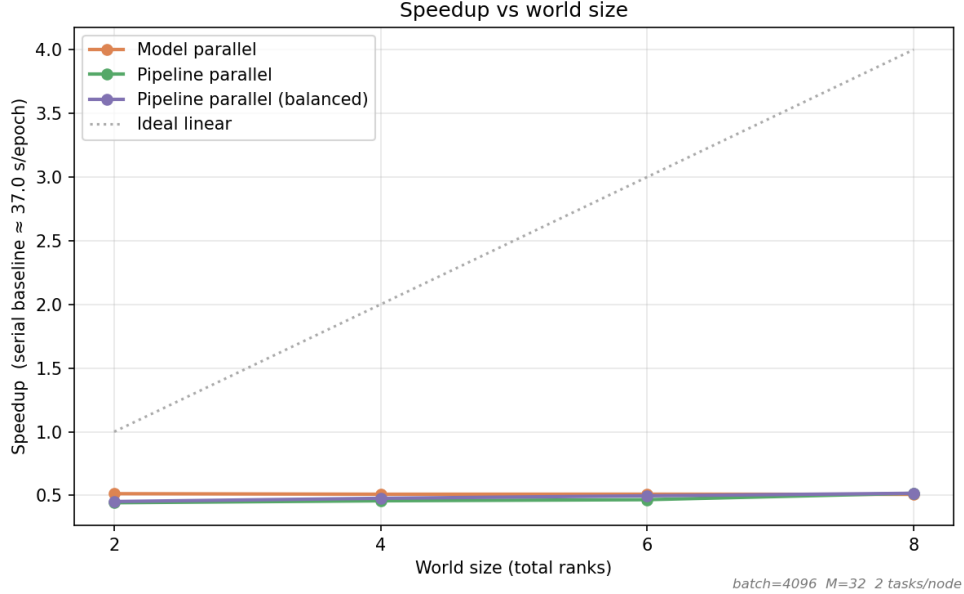


Figure 7: Speedup vs. serial baseline for model-parallel modes (batch = 4096, $M = 32$, 2 tasks/node). All modes remain at $\approx 0.5\times$, indicating that model parallelism at this scale imposes a net overhead relative to serial.

The positive result is that the load-balanced partitioner consistently outperforms naive round-robin when world size does not evenly divide the layer count, and the pipeline schedule with a small microbatch count ($M=8$) can match serial epoch time at batch = 4096. The negative result is that neither strategy overcomes the fundamental overhead of point-to-point activation exchange at this model and hardware scale. These findings are consistent with prior work suggesting that model parallelism is most beneficial when individual layers are too large to fit on a single device [Narayanan et al., 2019], a condition not met by this MNIST MLP.

5 Challenges and Obstacles

[Challenge 1]. TODO: describe what you tried, what happened, and your hypothesis for why.

[Challenge 2]. TODO.

[Challenge 3]. TODO.

References

- HPE Cray Programming Environment. intro_mpi documentation. https://cpe.ext.hpe.com/docs/25.03/mpt/mpich/intro_mpi.html, 2025. Accessed: 2026-05-12.
- Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Gregory Ganger, Phillip Gibbons, and Matei Zaharia. PipeDream: Fast and efficient pipeline parallel DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.

NERSC. Cray mpich. <https://docs.nersc.gov/development/programming-models/mpi/cray-mpich>, 2026a. Accessed: 2026-05-12.

NERSC. Network (perlmutter slingshot interconnect). <https://docs.nersc.gov/performance/network>, 2026b. Accessed: 2026-05-12.

Alexander Sergeev and Mike Del Balso. Horovod: Fast and easy distributed deep learning in TensorFlow. In *Proceedings of the 31st Conference on Neural Information Processing Systems Workshop*, 2018.

Sebastian U. Stich. Local SGD converges fast and communicates little. In *International Conference on Learning Representations*, 2019.